A Linear Algorithm for the Connected Domination Problem on Circular-Arc Graphs

Ruo-Wei Hung and Maw-Shang Chang
Department of Computer Science and Information Engineering
National Chung Cheng University, Ming-Hsiung, Chiayi 621, Taiwan, R.O.C.
{rwhung,mschang}@cs.ccu.edu.tw

Abstract

A connected domination set of a graph is a set D of vertices such that every vertex not in D is adjacent to at least one vertex in D, and the induced subgraph of D is connected. Given a circular-arc graph G in arc model with n sorted arcs, we present an algorithm for finding a minimum connected domination set of G. Our algorithm runs in $O(n)$ time and space.

1 Introduction

Let $G = (V, E)$ be an undirected simple graph. Throughout this paper, n and m denote the numbers of vertices and edges of a graph, respectively. For a vertex $v \in V$, let $N(v) = \{u \mid u \in V, u \neq v$ and u is a neighbor of $v\}$ and $N[v] = N(v) \cup \{v\}$. In general, for $S \subseteq V$, $N(S)$ and $N[S]$ denote $\cup_{v \in S} N(v)$ and $N(S) \cup S$, respectively. A subset S of V dominates another subset R of V if $R \subseteq N[S]$. A domination set S of a subset R of V is called a *connected domination set* of R if the induced subgraph of S is connected. A subset D of V is called a *connected domination set* of graph G if D is a *connected* domination set of V. Domination sets and their variants have applications in a variety of fields, including communication theory and political science. For more background on domination sets and their variants, we refer the reader to [8] and [9].

The connected domination problem is to find a minimum cardinality connected domination set of a graph. This problem is NP-hard in general graphs [6]. The same holds true even restricted to chordal graphs [15], circle graphs [11], weighted co-comparability graphs [3], etc. However, there exist polynomial algorithms to solve the connected domination problem on some special classes of graphs [2][13][16][20].

A graph G is an *interval graph* if there is a one-to-one correspondence between the vertices of G and the intervals of an interval family I such that two vertices of G are adjacent if and only if their corresponding intervals overlap. The interval family I is called an interval model of G. For an interval family I, $G(I)$ denotes the graph constructed from I. A circular arc family F is a collection of arcs on a circle. A graph G is a *circular-arc graph* if there is a circular arc family F and a one-to-one mapping of the vertices of G and the arcs in F such that two vertices in G are adjacent if and only if their corresponding arcs in F overlap. The circular-arc family F is called an arc model of the graph. For an arc family F, $G(F)$ denotes the graph constructed from F. If there exists a point on the circle such that no arc of F passes through, then $G(F)$ is an interval graph.

Tucker [19] presented an $O(n^3)$ time algorithm for recognizing a circular-arc graph. An arc model F can be built as a byproduct of this recognition algorithm in the affirmative case. Eschen and Spinrad [4] proposed an $O(n^2)$ time recognition algorithm for circular-arc graphs. Hence, researchers who study circular-arc graphs sometimes assume that a set of endpoints of sorted arcs is given [10][14]. If endpoints are not sorted, then they can be sorted in $O(nlogn)$ time. Thus, we assume that the endpoints of F are sorted. Circular-arc graphs have a variety of applications involving traffic light sequencing, genetics, etc. They have been studied in [2][7].

Researchers have investigated the algorithmic complexities of domination and its variants on circular-arc graphs. Hsu and Tsai [10] presented a linear-time algorithm to solve the domination problem on circular-arc graphs. Recently, Chang [2] proposed $O(n+m)$ algorithms to solve the domination and connected domination problems on circular-arc graphs with weights on vertices (arcs).

In this paper, we present an $O(n)$ time and

space algorithm for finding a minimum cardinality connected domination set on a circular-arc graph in an arc model with endpoints of n arcs sorted. To construct our algorithm in $O(n)$ time and space, we will solve the *query tree* problem defined as follows: Given a static rooted tree T of n nodes and a positive integer d, find all pairs $(u, \mathcal{A}_d(u))$ for each node u with a level number greater than or equal to d in T where $\mathcal{A}_d(u)$ is the d-th ancestor of u in T.

This paper is organized as follows: In Section 2, we present an algorithm to solve the query tree problem. Section 3 proposes an algorithm to solve the connected domination problem on circular-arc graphs. The above two algorithms run in $O(n)$ time and space. In Section 4, we prove that the complexity of the algorithm in Section 3 is $O(n)$ time and space. Finally, we make some concluding remarks in Section 5.

2 The Query Tree Problem

To solve the *query tree* problem, we can visit d ancestors of each node in a static rooted tree T and obtain the corresponding ancestor at a time. Thus, the total time of this trivial algorithm will be $O(dn)$. In this section, we use a different strategy to solve it in $O(n)$ time.

For each node $u \in T$, denote by $p(u)$ and $\mathcal{A}_d(u)$ the parent node and the d-th ancestor of u, respectively. Note that $p(u) = \mathcal{A}(u)$ if they exist. Furthermore, let $T(u)$ denote the subtree of T rooted at u and $l(u)$ denote the level number of u.

Definition 1. Define $l(u) = 0$ if u is the root of T and $l(u) = l(x) + 1$ if u is not the root of T and $x = p(u)$. Define the height of T, denoted by $h(T)$, to be $max\{l(y)|$ for all $y \in T\}$.

In the following, we briefly describe the static disjoint set union and find algorithm proposed by Gabow and Tarjan [5] that is used in our algorithm: The disjoint set union-find problem [1][5] is to carry out the following three kinds of operations on disjoint sets:
$makeset(x)$: Create a new singleton set $\{x\}$ whose name is x for an element found in no existing set.
$find(x)$: Return the name of the set currently containing element x.
$union(x, y)$: Combine the sets containing elements x and y into a new set whose name is the name of the old set containing element x. This operation requires that x and y are initially in different sets.

The fastest known algorithm for this problem runs in $O(p\alpha(p + q, q) + q)$ time and $O(q)$ space, where α is an inverse of Ackermann's function [17][18], q is the number of elements, and p is the number of operations performed. A special case of the disjoint set union-find problem can be formalized as follows: We are given a tree T of q nodes that correspond to the initial q singleton sets. The problem involves performing a sequence of *union* and $find$ operations such that each *union* can be only of the form $union(p(u), u)$. T is called the *static union tree* and the problem is referred to as the *static tree set union* problem. For this special case, each *union* and *find* operation can be supported in $O(1)$ amortized time on a random access machine, and the total space required is $O(q)$ [5].

We then sketch our algorithm for the query tree problem below. First, our algorithm finds $l(u)$ for each node $u \in T$ by a *breadth-first search*. The height $h(T)$ of T can be computed easily. Then, we use a bottom-up approach to compute $\mathcal{A}_d(u)$ for each $u \in T$ by the static disjoint set union and find algorithm. Let u be the node currently visited. If $l(u) \geq (h(T) - d + 1)$, then it performs only the $union(p(u), u)$ operation. Otherwise, it first performs a sequence of $find(w)$ operations and outputs $(w, \mathcal{A}_d(w))$ for node w in $T(u)$ with $l(w) = l(u) + d$, then it performs $union(p(u), u)$. After completing the traversal, our algorithm terminates.

We give an example to illustrate our algorithm below.

Example 1. Given a static tree T with root v shown in Figure 1 and $d = 2$. First, our algorithm visits the nodes of T by breadth-first traversal and obtains that $h(T) = 5$. Then, it visits the nodes in T by a bottom-up traversal. When the node v with $l(v) \geq (h(T) - d + 1) = 4$ is visited, it performs $union(p(v), v)$ only. Hence, our algorithm performs a sequence of *union* operations which are $union(v_{12}, v_{15})$, $union(v_{12}, v_{14})$, $union(v_9, v_{13})$, $union(v_8, v_{12})$, $union(v_5, v_{11})$, $union(v_5, v_{10})$, and $union(v_5, v_9)$. When the node v with $l(v) < 4$ is visited, it performs a sequence of *find* operations. Assume that v_8 is the node currently visited. Our algorithm performs a sequence of *find* operations which are $find(v_{15})$ and $find(v_{14})$. Then, it sets $\mathcal{A}_2(v_{15}) = find(v_{15}) = v_8$ and $\mathcal{A}_2(v_{14}) = find(v_{14}) = v_8$, and outputs $(v_{15}, \mathcal{A}_2(v_{15}))$ and $(v_{14}, \mathcal{A}_2(v_{14}))$. Then, it performs $union(v_4, v_8)$. Finally, our algorithm terminates after visiting the root and performing a sequence of *find* operations at level $2 = d$. ■

We use the static disjoint set union and find

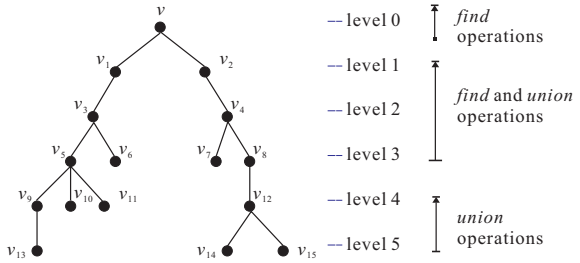

Figure 1: The *union* and *find* operations for the query tree problem in a rooted tree.

algorithm [5] to solve the *query tree* problem. Denote our algorithm and its output by Algorithm QT(T, d) and QT, respectively, where T and d are the input of Algorithm QT. Then, each *union* and *find* operation can be supported in $O(1)$ amortized time on a random access machine [5] since the input is a static rooted tree. It is easy to see that the number of *find* and *union* operations performed in Algorithm QT does not exceed $2n$. Furthermore, all other operations can be done in constant time, and the size of all data structures used never exceeds $O(n)$. Thus, Algorithm QT runs in $O(n)$ time and space. The following result immediately holds:

Theorem 2.1. *Given a rooted tree T with n nodes and an integer d, Algorithm* QT *computes QT in $O(n)$ time and space, where QT contains all pairs $(u, \mathcal{A}_d(u))$ for $u \in T$ and $l(u) \geq d$.*

3 The Algorithm for the Connected Domination Problem

In this section, we present a linear algorithm for finding a minimum cardinality connected domination set on a circular-arc graph. Recall that we have an arc family F. Denote an arc i that begins at endpoint $h(i)$ and ends at endpoint $t(i)$ in the clockwise direction by $(h(i), t(i))$. Define $h(i)$ and $t(i)$ to be the *head* (*counterclockwise endpoint*) and *tail* (*clockwise endpoint*) of arc i, respectively. Without loss of generality, assume that all endpoints are distinct and no arc covers the entire circle. Label the n arcs from 1 through n such that $h(i) < h(j)$ if $i < j$.

The continuous part of the circle that begins with a point s and ends with t in the clockwise direction is referred to as segment (s, t), denoted by $seg(s, t)$, of the circle. We use "*arc*" to refer to a member of F and "*segment*" to refer to a part of the circle between two points. A point p on the circle is said to be in arc (segment) (c, d) if it falls within the interior of $seg(c, d)$. Denote this by $p \in seg(c, d)$. Arc i is contained in arc j if every point of arc i is in arc j.

Given an arc family F, let F' be the maximal collection of arcs which are not contained in any other arc of F. We call F' the proper arc-family of $G(F)$ with respect to F. Notice that if a minimum-cardinality connected domination set D contains arc $i \in F - F'$, then $(D - \{i\}) \cup \{j\}$ is still a minimum-cardinality connected domination set where $j \in F'$ and j contains i. On the other hand, if D is a minimum connected domination set restricted to F' on $G(F)$, then it is also a minimum connected domination set of $G(F)$. Hence, it is easy to show that the solution for the connected domination problem on $G(F)$ can be restricted to F'.

Definition 2. Define $N_c(i) = \{j \mid j \in N(i),\ j \in F' \text{ and } t(i) \in j\}$. Define the *successor arc* function χ, which maps an arc into another arc or 0, as follows: For $i \in F'$, $\chi(i) = 0$ if there exists no arc j such that $j \in N_c(i)$; otherwise, $\chi(i) = j$ if $j \in N_c(i)$ and $seg(t(i), t(k)) \subseteq seg(t(i), t(j))$ for all arcs $k \in N_c(i)$.

Throughout this paper, we use the abbreviation $\chi_j(i)$ to represent $\overbrace{\chi(\chi(\cdots\chi}^{j}(i))\cdots)$ for $j \geq 2$. That is, $\chi_j(i) = \chi_{j-1}(\chi(i))$. Note that if there exists some arc $i \in F'$ such that $\chi(i) = 0$, then the graph is an interval graph. In this case, we can solve the connected domination problem in $O(n)$ time and space using Chang's algorithm [2]. Hence, we assume that $\chi(i) \neq 0$ for $i \in F'$ in the remainder of the paper.

Definition 3. Let i be an arc in F'. A *circle sequence* starting with arc i, denoted by $Cir(i)$, is a sequence of arcs $i, \chi(i), \cdots, \chi_p(i)$ where p is the smallest integer such that $t(\chi_p(i))$ is in arc i.

Since there exists no arc that covers the entire circle, it is clear that $|Cir(i)| \geq 2$ for all $i \in F'$. By definition of circle sequence, $Cir(i)$ is a connected domination set for any arc $i \in F'$ since it covers the entire circle and is a connected subgraph of $G(F)$.

Definition 4. Let i be an arc in F'. A *cycle sequence* starting with arc i, denoted by $Cyc(i)$, is a sequence of arcs $i, \chi(i), \cdots, \chi_q(i)$ where q is the smallest integer such that $\chi_{q+1}(i) = i$.

Notice that there might not exist a cycle sequence starting from some arc i in F'. In this case, we say that $Cyc(i) = \varnothing$. Obviously, if $Cyc(i) \neq \varnothing$, then $|Cyc(i)| \geq 2$ and $Cir(j) \subseteq Cyc(i)$ for all $j \in Cyc(i)$. For simplicity, let $\mathcal{C}_0(i) = \{i\}$ and $\mathcal{C}_q(i) = \{i, \chi(i), \cdots, \chi_q(i)\}$ for $q > 0$.

In the following, we prove some theorems which are the bases of our linear algorithm for the connected domination problem on circular-arc graphs.

Theorem 3.1. *If circular-arc graph $G(F)$ has a connected domination set, then there exist a minimum-cardinality connected domination set S, an arc $i \in F'$ and an integer q that satisfy all of the following three statements:*
(1) $S = \mathcal{C}_q(i)$;
(2) $0 \leq q \leq |Cir(i)| - 1$;
(3) *if* $|Cir(i)| \geq 4$, *then* $q \geq |Cir(i)| - 3$.

Proof. First, we prove that there exists a set $MCDS$ that is a subset of F'. Let $MCDS$ be a minimum connected domination set such that $|MCDS - F'|$ is minimum. Suppose $MCDS - F' \neq \varnothing$. Let arc $u \in MCDS - F'$. Then, there exists another arc v that contains u. Apparently, $(MCDS - \{u\}) \cup \{v\}$ is also a minimum connected domination set. This contradicts the assumption. Hence, $MCDS - F' = \varnothing$.

If $MCDS$ covers the entire circle, then let arc i be any arc in $MCDS$. Otherwise, let arc i be the arc with the most counterclockwise head which is not contained in any arc in $MCDS$. Then, starting from arc i, label the arcs of $MCDS$ from 0 through $|MCDS| - 1$ in the clockwise order of clockwise endpoints. Let j be the smallest integer such that the j-th arc of $MCDS$ which is not in $Cir(i)$ (i is the 0-th arc of $MCDS$). Replace this arc with $\chi_j(i)$. The new set is still a minimum connected domination set. By repeating the above process, we finally obtain an $MCDS$ which is a subset of $Cir(i)$. Hence, statements (1) and (2) are correct.

It is easy to see that $\mathcal{C}_q(i)$ is not a connected domination set if $q \leq |Cir(i)| - 4$. Hence, statement (3) is also correct. **Q.E.D.**

Lee et al. [12] proved that if $|Cir(v)| - |Cir(\mu)| = 1$ for $\mu \neq v$, then $Cir(\mu)$ is the minimum cardinality set that covers the entire circle. This implies that $|Cir(v)|$ is at most one more than the size of the minimum circle sequence for all $v \in F'$. Figure 2 depicts such an example, where $Cir(2)$ is the minimum circle sequence of size 3 and $|Cir(v)| \leq 4$ for all $v \in F'$. Thus, we have the following theorem:

Theorem 3.2. [12] *If there exists no arc w such that $\chi(w) = 0$ and $Cir(\mu)$ is the minimum cardinality set to cover the entire circle, then $|Cir(v)| \leq |Cir(\mu)| + 1$ for all $v \in F'$.*

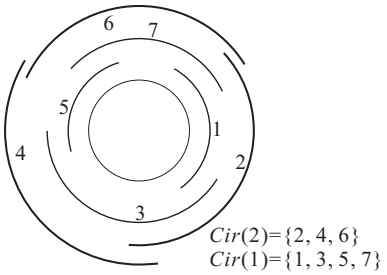

Figure 2: An example of Theorem 3.2: $|Cir(v)| \leq |Cir(2)| + 1$, for all $v \in F'$.

Theorem 3.3. *If $Cir(\mu)$ exists for some $\mu \in F'$, then $|Cir(\mu)| - 1 \leq |Cir(v)| \leq |Cir(\mu)| + 1$ for all $v \in F'$.*

Proof. Assume that arc μ is in the minimum cardinality circle sequence. By Theorem 3.2, we have $|Cir(v)| \leq |Cir(\mu)| + 1$ for all $v \in F'$. Now, suppose that arc μ is not in the minimum cardinality circle sequence. The size of $Cir(\mu)$ is at most one more than the size of the minimum circle sequence by Theorem 3.1. Thus, $|Cir(\mu)| \leq |Cir(v)| + 1$ where $Cir(v)$ is the minimum circle sequence. This implies that $|Cir(\mu)| - 1 \leq |Cir(v)|$ where $|Cir(v)|$ is the size of the minimum circle sequence. This completes the proof. **Q.E.D.**

In [10], Hsu and Tsai proved that the element of the minimum domination set on $G(F)$ can be restricted to be in one cycle sequence. One conjectures whether the same property can be applied to the connected domination problem on circular-arc graphs. If it is true, then we can save a lot of trouble in solving the connected domination problem. But this conjecture is not true. For instance, the minimum cardinality connected domination set $MCDS$ is $\{2, 5, 8\}$ in Figure 3. Any element in $MCDS$ is not in the cycle sequence $\{1, 4, 7, 10\}$.

Following Theorem 3.1, Theorem 3.2 and Theorem 3.3, we propose a linear algorithm for finding a minimum-cardinality connected domination set on a circular-arc graph. We first sketch our algorithm as follows: First, our algorithm extracts F' from F. Then, it computes $\chi(i)$ for each $i \in F'$. If there exists one arc i such that $\chi(i) = 0$, then $G(F)$ is

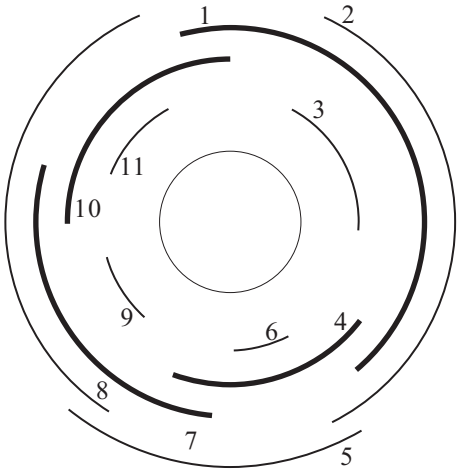

Figure 3: An arc family F.

an interval graph. In this case, we can solve the connected domination problem by Chang's algorithm [2] in $O(n)$ time and space. In the following, assume that there exists no arc whose successor arc is empty. Then, it picks an arbitrary arc $\mu \in F'$ and computes the size γ of $Cir(\mu)$. By Theorem 3.3, $\gamma - 1 \leq |Cir(v)| \leq \gamma + 1$, for all $v \in F'$. By Theorem 3.1, it is easy to see that $\mathcal{C}_q(\mu)$ is not a domination set if $q \leq \gamma - 4$. Hence, $\mathcal{C}_p(v)$ is not a domination set if $p \leq \gamma - 5$ for all $v \in F'$. Hence, our algorithm finds the largest $\jmath$ such that $\mathcal{C}_{\gamma-\jmath}(v)$ is a domination set for $1 \leq \jmath \leq 4$. Note that $\mathcal{C}_{\gamma-\jmath}(v)$ is a connected subgraph of $G(F)$ and is a domination set of $G(F)$ if there exists no arc i in F such that arc i is contained in $seg(t(\chi_{\gamma-\jmath}(v)), h(v))$.

We formally present our linear algorithm for finding a minimum cardinality connected domination set below and analyze it in Section 4.

Algorithm MCCDS. Find a minimum cardinality connected domination set of $G(F)$.
Input: A set F of sorted arcs where each arc i is represented as $(h(i), t(i))$.
Output: A set $MCDS$ of arcs such that $MCDS$ is a minimum-cardinality connected domination set of $G(F)$.
Method:

1. Extract F' from F;
2. Compute the successor arc $\chi(i)$ for each arc $i \in F'$;
3. **if** $\chi(i) = 0$ for some $i \in F'$, **then** Call Chang's algorithm [2] and **Stop.**
4. Pick an arc $u \in F'$ arbitrarily and compute $|Cir(u)|$;
5. $\gamma \longleftarrow |Cir(u)|$;
6. **for** $\jmath = 4$ downto 1, **do**
7. **if** $\jmath = 1$,
8. **then** $MCDS \longleftarrow Cir(u)$; **Output** $MCDS$;
9. **if** $\gamma - \jmath \geq 0$, **then**
10. Compute $\chi_{\gamma-\jmath}(v)$ for all $v \in F'$;
11. **if** $\mathcal{C}_{\gamma-\jmath}(v)$ is a domination set for $v \in F'$,
12. **then** $MCDS \longleftarrow \mathcal{C}_{\gamma-\jmath}(v)$;
13. **Output** $MCDS$;

Now, we give an example to illustrate Algorithm MCCDS below.

Example 2. Assume F is the set of arcs given in Figure 4. We first find F' shown in Figure 5. Then we compute the successor arc function χ for all arcs in F'. That is, $\chi(1) = 6, \chi(3) = 7, \chi(4) = 8, \chi(5) = 8, \chi(6) = 9, \chi(7) = 11, \chi(8) = 11, \chi(9) = 13, \chi(11) = 13, \chi(12) = 15, \chi(13) = 1$, and $\chi(15) = 1$. In this time, $\chi(i) \neq 0$ for $i \in F'$. Hence, $G(F)$ is not an interval graph. Pick an arbitrary arc called arc 1 and compute $|Cir(1)| = 4 = \gamma$. Then we find the largest integer $\jmath$, $1 \leq \jmath \leq 4$, such that there exists an arc v, and $\mathcal{C}_{4-\jmath}(v)$ is a domination set. In the iterations of $\jmath = 4$ or 3, it is not difficult to see that $\mathcal{C}_{4-\jmath}(v)$ is not a domination set for each $v \in F'$. In the iteration of $\jmath = 2$, we have that $seg(t(\chi_2(9)), h(9))$ contains no arc in F where $\chi_2(9)$ is arc 1. Hence, $\mathcal{C}_2(9) = \{9, 13, 1\}$ is a connected domination set of $G(F)$. Then, Algorithm MCCDS terminates at $\jmath = 2$ and outputs $MCDS = \mathcal{C}_2(9) = \{9, 13, 1\}$ which is a minimum cardinality connected domination set of $G(F)$ in Figure 4. ■

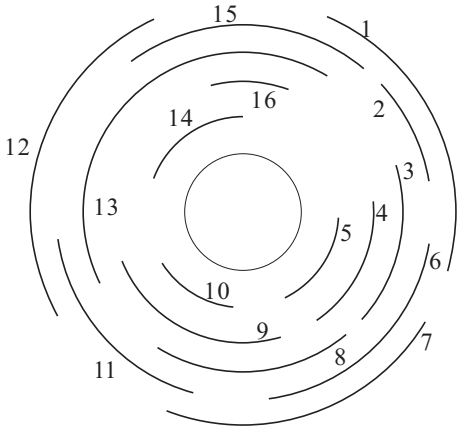

Figure 4: An arc family F with endpoints sorted.

Following Theorem 3.1, Theorem 3.2 and Theorem 3.3, the following result immediately holds:

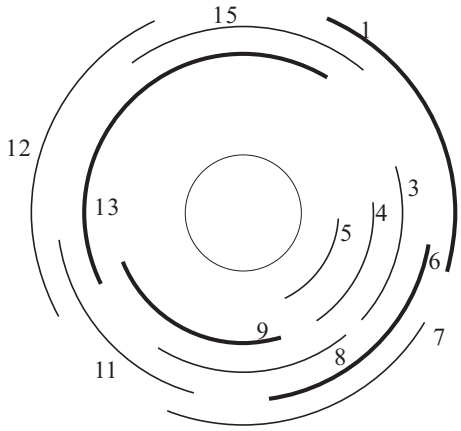

Figure 5: A proper arc-family F' with respect to F in Figure 4.

Theorem 3.4. *Given a sorted arc family F, Algorithm* MCCDS(F) *outputs a minimum-cardinality connected domination set on $G(F)$.*

4 The Correctness and Complexity

The correctness of Algorithm MCCDS follows Theorem 3.1, Theorem 3.2 and Theorem 3.3. In this section, we analyze the complexity of Algorithm MCCDS.

Extract F' from F and compute their successor arcs that can be determined in $O(n)$ time [12]. Thus we have the following two lemmas:

Lemma 4.1. *Given a sorted arc family F, F' can be found in $O(n)$ time and space.*

Lemma 4.2. *Given a proper-arc family F', $\chi(v)$'s can be computed, for all $v \in F'$, in $O(n)$ time and space.*

The remaining steps we consider will be only line 10 and line 11 of Algorithm MCCDS. Given $\chi(v)$'s and a positive integer d, we present a linear algorithm to find $\chi_d(v)$'s for all $v \in F'$ in the following.

Definition 5. Let F be an arc family and F' be the proper proper-arc family with respect to F. Define $G^* = (V^*, E^*)$ to be a directed graph obtained from F' where $V^* = F'$ and $(i, j) \in E^*$ if and only if $\chi(i) = j$.

Obviously, the out-degree of each node in V^* is 1 since the successor arc of each arc in F' is *unique*. Note that a cycle sequence in F' is a simple cycle in G^*, and no two simple cycles can share a common vertex. For instance, Figure 6 depicts the graph G^* constructed from F' in Figure 5. We use the notions "node in G^*" and "arc in F'" interchangeably. Furthermore, we can use auxiliary pointer to connect one node in G^* and it corresponding arc in F'. By the definition of G^*, the following propositions are not difficult to verify:

Proposition 1. *There exists at least one simple cycle in G^* if $G(F)$ is not an interval graph.*

Proposition 2. *Any two distinct simple cycles of G^* are pairwise disjoint if they exist.*

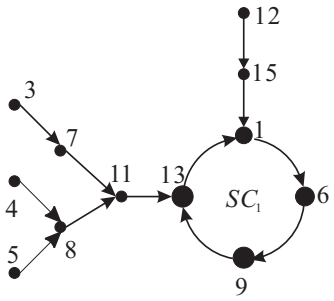

Figure 6: The graph G^* constructed from F' in Figure 5.

By Proposition 1, there exists at least one simple cycle in G^* if $G(F)$ is not an interval graph. By Proposition 2, there might exist more than one connected component and two distinct components are disjoint in $G^* = (V^*, E^*)$. Assume that G^* has k disjoint connected components. Then, each connected component is a *1-tree*, denoted by $D_\imath$, $1 \le \imath \le k$, which is the union of a static tree and an edge connecting two nodes in this tree. Therefore, there exists just one simple cycle, denoted by $SC_\imath$, in $D_\imath$. An *in-neighbor* of a node v is a node u if $(u, v) \in E^*$, and an *out-neighbor* of a node v is a node w if $(v, w) \in E^*$. If we discard two edges (v, w) and (u, v) from SC_i, we will obtain two connected subgraphs of $D_\imath$. One is a rooted directed tree $T(v)$ with root $v \in SC_\imath$, called the *pendant tree* of $SC_\imath$. Note that $D_\imath$ consists of one simple cycle and $|SC_\imath|$ pendant trees.

We then sketch our linear algorithm for finding all $\chi_d(v)$'s as follows: Our algorithm first computes all connected components $D_\imath$ of G^*, $1 \le \imath \le k$. Then, it finds all simple cycles $SC_\imath$ in $D_\imath$, for $\imath = 1$ to k, and computes all pendant trees $T_{\imath j}$ by discarding all edges in $SC_\imath$, for $j = 1$ to $|SC_\imath|$. Note that $D_\imath = SC_\imath \cup T_{\imath j}$ and $|T_{\imath j}| \ge 1$ for $1 \le j \le |SC_\imath|$. Then, it relabels the nodes in $SC_\imath$

from 0 to $|SC_i| - 1$ sequentially. Since the nodes in simple cycle SC_i are circular, our algorithm finds $\chi_d(v)$ by using the *modulus* operation for each node $v \in SC_i$. Then, it processes the nodes in pendant trees. There are two kinds of nodes in pendant trees. One consists of the nodes whose level numbers are less than d. The other type of nodes contains the nodes with level number $\geq d$. We call the former type of nodes Type $\mathbb{I}$ and the latter one Type $\mathbb{III}$. For the node v in Type $\mathbb{I}$, our algorithm applies the *modulus* operation to find its d-th successor node since $\chi_d(v)$ is in some simple cycle of G^*. For all nodes v in Type $\mathbb{III}$, it calls Algorithm QT presented in Section 2 to find their d-th ancestor nodes. We formally present the above process in the following:

Algorithm Find. Find $\chi_d(v)$*'s* for all $v \in D_i$.
Input: A connected component D_i of G^* and a positive integer d.
Output: A set $X = \{(v, w)\}$, where $\chi_d(v) = w$ for all $w, v \in D_i$.
Method:

1. Compute the simple cycle SC_i of D_i;
2. $\eta \longleftarrow |SC_i|$;
3. Pick a vertex $u \in SC_i$ arbitrarily and let $\varphi(u) = 0$;
4. **for** all $(v_i, v_j) \in SC_i$ and $v_j \neq u$, **do** $\varphi(v_j) = \varphi(v_i) + 1$;
5. Compute all pendant trees T_{ij} of SC_i, for $1 \leq j \leq \eta$;
6. $X \longleftarrow \varnothing$;
7. **for** all $v \in SC_i$, **do**
8. $\mu \longleftarrow (\varphi(v) + d) \bmod \eta$;
9. Let w be the node in SC_i where $\varphi(w) = \mu$;
10. $X \longleftarrow X \cup \{(v, w)\}$;
11. **for** $j = 1$ to η, **do**
12. Let v_j be the root of T_{ij};
13. Compute $l(v)$ for each $v \in T_{ij}$;
14. $h(T_{ij}) \longleftarrow max\{l(v) \mid v \in T_{ij}\}$;
15. **if** $h(T_{ij}) \geq d$, **then**
16. Call Algorithm QT(T_{ij}, d) and let QT be its output;
17. $X \longleftarrow X \cup QT$;
18. **for** each $v \in T_{ij}$ with $l(v) < d$, **do**
19. $\mu \longleftarrow (\varphi(v_j) + d - l(v)) \bmod \eta$;
20. Let w be the node in SC_i where $\varphi(w) = \mu$;
21. $X \longleftarrow X \cup \{(v, w)\}$;
22. **Output** X.

We give an example to illustrate Algorithm Find in the following:

Example 3. Given a 1-tree shown in Figure 7 and $d = 2$. The annotation of node v in Figure 7 is that if v is in simple cycle, then $(a, b(c))$ denotes $(v, \varphi(v)(l(v)))$; otherwise, $(a(c))$ denotes $(v(l(v)))$. If node v is in either simple cycle or Type $\mathbb{I}$, then we find $\chi_2(v)$ by the modulus operation. For instance, given node 9, we can find that $\chi_2(9)$ is node 1 where $\varphi(9) = 2$ and $\varphi(1) = (\varphi(9) + 2) \bmod 4 = 0$. If we are given node 11, then the root of the pendant tree containing node 11 is node 13. Hence, $\varphi(13) + 2 - l(11) \bmod 4 = 0$. We can easily see that $\varphi(1) = 0$. That is, node 1 is the second successor node of node 11. Thus, we have that $\chi_2(11)$ is node 1. We then call Algorithm QT(T_1, 2) and Algorithm QT(T_2, 2) where T_1 and T_2 are the pendant trees rooted at nodes 13 and 1, respectively. Finally, we can obtain a set $X = \{(v, w)\}$ where $\chi_2(v) = w$ for all v in D_i. ■

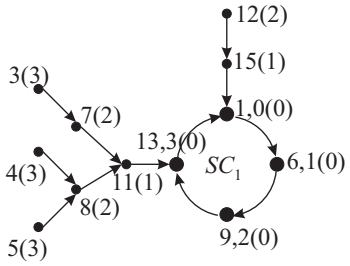

Figure 7: A 1-tree D_i and its corresponding information.

Since the out-degree of each node in $G^* = (V^*, E^*)$ is at most one, we have $|E^*| \leq |V^*|$. We then have the following lemma:

Lemma 4.3. *The construction of G^* with the aforementioned computation of $\chi(\cdot)$ requires $O(n)$ time and space.*

Lemma 4.4. *Line 10 of Algorithm* MCCDS *can be computed in $O(n)$ time and space.*

Proof. There might exist more than one connected component in $G^* = (V^*, E^*)$, and two distinct components are disjoint. Recall that a connected component D_i contains a simple cycle SC_i and some pendant trees.

For each node v in either SC_i or Type $\mathbb{I}$, we can find $\chi_{\gamma-J}(v)$ by using the *modulus* operation in $O(1)$ time. For all nodes $w \in$ Type $\mathbb{III}$, we can call Algorithm QT presented in Section 2 to find $\chi_{\gamma-J}(w)$*'s*. Following Theorem 2.1, this can be done in $O(|D_i|)$ time and space. Thus, Algorithm

Find can be done in $O(|D_i|)$ time and space. That is, line 10 of Algorithm MCCDS can be done in $O(|D_1|+|D_2|+\cdots+|D_k|) = O(n)$ time and space. This completes the proof. **Q.E.D.**

In the following, we show that line 11 of Algorithm MCCDS can be done in $O(n)$ time and space:

After computing $\chi_d(v)$ for each $v \in F'$, we can construct fictitious arcs $\tilde{v}$ as $seg(t(\chi_d(v)) + \epsilon, h(v) - \epsilon)$ for each $v \in F'$, where ϵ is a tiny number. To distinguish fictitious arcs from real arcs in F, we label fictitious arc $\tilde{v}$ to be $v + n$ where $\tilde{v} = seg(t(\chi_d(v)) + \epsilon, h(v) - \epsilon)$. We then construct a new arc family $\mathcal{F} = F \cup \{\tilde{v}\}$, where $\tilde{v}$ are fictitious arcs. For instance, Figure 8 depicts the new arc family $\mathcal{F}$ constructed from Figure 4 and $d = 2$.

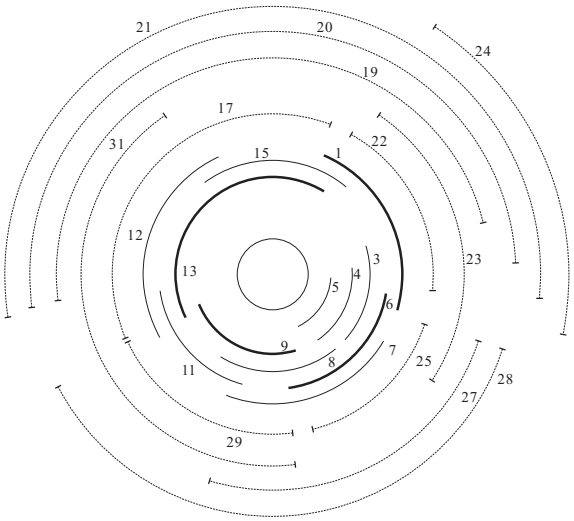

Figure 8: An arc family $\mathcal{F} = F \cup \{seg(t(w), h(v))\}$, where $\chi_2(v) = w$.

Now, we point out the implement in line 11 of Algorithm MCCDS. Recall that we have two kinds of arcs in $\mathcal{F}$. One contains the arcs in F, and the other consists of the fictitious arcs with label numbers $> n$. We first scan the endpoints of arcs in $\mathcal{F}$ twice in a clockwise direction and mark all arcs including fictitious arcs containing any arc of F. Then, we scan the members of $\mathcal{F}$. If there exists some fictitious arc $\tilde{v} \in \mathcal{F} - F$ such that $\tilde{v}$ is not marked, then fictitious arc $\tilde{v}$ contains no arc in F. That is, $\mathcal{C}_d(v)$ is a domination set of $G(F)$ where v is the corresponding arc of $\tilde{v}$ and $v = \tilde{v} - n$.

When we scan the endpoints of $\mathcal{F}$, we can use queue Q to store some information. If the endpoint traversed is a counterclockwise endpoint of some arc i, then insert $h(i)$ into the tail of Q. Otherwise, we inspect whether or not arc i is a fictitious arc. If it is not a fictitious arc and any other $h(j)$ happens to be ahead of $h(i)$ in Q, then mark j and remove $h(j)$ from Q. Finally, we remove $h(i)$ from Q when a clockwise endpoint $t(i)$ is encountered.

In the following, we prove that the above procedure can be applied to solve line 11 of Algorithm MCCDS correctly. Since $h(y)$ of the marked arc y is ahead of $h(x)$ of $x \in F$ on Q and $t(x)$ of x is encountered before $t(y)$ of y, the marked arc y contains x. Now, suppose there exist arcs i and j such that $i \in F$ and j contains i in the unmarked arc family. Then, either $h(j)$ is encountered ahead of $h(i)$ or $h(j)$ is encountered behind of $h(i)$ in the first scan of the circle. If $h(j)$ is encountered ahead of $h(i)$ in the first scan, then the algorithm will mark j, a contradiction. In another case, j is not marked but $h(i)$ is removed from Q. In the second scan, $h(j)$ will be in Q. When $h(i)$ is encountered, it is placed at the rear of $h(j)$ in Q. Since $t(i)$ is encountered ahead of $t(j)$ in the second scan, arc j will be marked when $t(i)$ is encountered, a contradiction. Thus, there do not exist such arcs i and j in the unmarked arc family. Hence, the process in the previous paragraph can be applied to inspect whether there exists a fictitious arc $\tilde{v}$ such that it contains no arc in F. That is, line 11 of Algorithm MCCDS can be solved correctly as shown in the previous paragraph. Since $|\mathcal{F}| \leq 2n$ and we scan the endpoints of $\mathcal{F}$ twice, the following result immediately holds:

Lemma 4.5. *Line 11 of Algorithm* MCCDS *can be determined in $O(n)$ time and space.*

Following the lemmas in this section and Theorem 3.4, we conclude the following theorem:

Theorem 4.6. *Algorithm* MCCDS *solves the connected domination problem on circular-arc graphs in $O(n)$ time and space.*

5 Concluding Remarks

In this paper, we present linear algorithms to solve the *query tree* problem on trees and the *connected domination set* problem on circular-arc graphs. It will be interesting to see if the same results can be applied to the variants of the domination problem on circular-arc graphs.

References

[1] A. V. Aho, J. E. Hopcroft and J. D. Ullman, *The Design and Analysis of Computer Algorithms*, Addision-Welsley, MA, 1974.

[2] M. S. Chang, *Efficient algorithms for the domination problems on interval and circular-arc graphs*, SIAM J. Comput. 27 (1998) 1671-1694.

[3] M. S. Chang, *Weighted domination of cocomparability graphs*, Discrete Appl. Math. 80 (1997) 135-147.

[4] E. M. Eschen and J. P. Spinrad, *An $O(n^2)$ algorithm for circular-arc graph recognition*, Proceedings of the Fourth Annual ACM-SIAM Symposium on Discrete Algorithm (1993) 128-137.

[5] H. N. Gabow and R. E. Tarjan, *A linear-time algorithm for a special case of disjoint set union*, J. Computer and Sys. Sci. 30 (1985) 209-221.

[6] M. R. Garey and D. S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*, W.H. Freeman, San Francisco, CA, 1979.

[7] M. C. Golumbic, *Algorithmic Graph Theory and Perfect Graphs*, Academic Press, New York, 1980.

[8] T. W. Haynes, S. T. Hedetniemi and P. J. Slater, *Fundamentals of Domination in Graphs*, Marcel Dekker, New York, 1998.

[9] T. W. Haynes, S. T. Hedetniemi and P. J. Slater, *Domination in Graphs−Advanced Topics*, Marcel Dekker, New York, 1998.

[10] W. L. Hsu and K. H. Tsai, *Linear time algorithms on circular-arc graphs*, Inform. Process. Lett. 40 (1991) 123-129.

[11] J. M. Keil, *The complexity of domination problems in circle graphs*, Discrete Appl. Math. 42 (1993) 51-63.

[12] C. C. Lee and D. T. Lee, *On a circle-cover minimization problem*, Inform. Process. Lett. 18 (1984) 109-115.

[13] Y. L. Lin, F. R. Hsu and Y. T. Tsai, *Efficient algorithms for the minimum connected domination on trapezoid graphs*, Lecture Notes in Comput. Sci. 1858, Springer-Verlag, New York, Berlin, 2000, pp. 126-136.

[14] S. Masuda and K. Nakajima, *An optimal algorithm for finding a maximum independent set of a circular-arc graph*, SIAM J. Comput. 17 (1988) 41-52.

[15] M. Moscarini, *Doubly chordal graphs, Steiner trees, and connected domination*, Networks 23 (1993) 59-69.

[16] C. Rhee, Y. D. Liang, S. K. Dhall and S. Lakshmivarahan, *An O(N+M)-time algorithm for finding a minimum-weight dominating set in a permutation graph*, SIAM J. Comput. 25 (1996) 404-419.

[17] R. E. Tarjan, *Efficiency of a good but not linear set union algorithms*, J. ACM 26 (1975) 215-225.

[18] R. E. Tarjan and J.van Leeuwen, *Worst-case analysis of set union algorithms*, J. ACM 31 (1984) 245-281.

[19] A. Tucker, *An efficient test for circular-arc graphs*, SIAM J. Comput. 9 (1980) 1-24.

[20] H. G. Yeh and G. J. Chang, *Weighted connected domination and Steiner trees in distance-hereditary graphs*, Discrete Appl. Math. 87 (1998) 245-253.